

Instruction Set Architecture (del processore Hack)

```
// Adds 1+...+100.
    @i      // i refers to some RAM location
    M=1     // i=1
    @sum    // sum refers to some RAM location
    M=0     // sum=0
(LLOOP)

    @i
    D=M     // D = i
    @100
    D=D-A   // D = i - 100
    @END
    D;JGT   // if (i-100) > 0 goto END
    @i
    D=M     // D = i
    @sum
    M=D+M   // sum += i
    @i
    M=M+1   // i++
    @LLOOP
    0;JMP   // goto LOOP
(END)
    @END
    0;JMP   // infinite loop
```

Instruction Set Architecture

- Rappresenta l'interfaccia fra hardware e software.
- Abbiamo studiato i principali componenti di un processore (circuiti logici combinatori e sequenziali, memorie, microarchitettura, ...).
- Ora dobbiamo studiare come questi vengono effettivamente usati.
- Per usare un processore, lo si deve programmare tramite un suo specifico **linguaggio assembly**, costituito da un insieme di istruzioni detto “**instruction set**” (i tipi di istruzioni disponibili nel linguaggio).
- Un linguaggio di tipo assembly è un tipo di linguaggio di basso livello (quindi non un linguaggio di alto livello usato comunemente da programmatori) in cui ogni istruzione di un programma corrisponde ad una istruzione macchina eseguibile dal processore.

Versione binaria e simbolica

- Le istruzioni in linguaggio macchina eseguibili da un processore sono codificate da sequenze binarie (versione “binaria”)
- L’Assembly ne è una versione leggibile (versione “simbolica”).
Esempio reale per Hack :

```
// Adds 1+...+100.
    @i      // i refers to some RAM location
    M=1     // i=1
    @sum    // sum refers to some RAM location
    M=0     // sum=0
(LLOOP)

    @i
    D=M     // D = i
    @100
    D=D-A   // D = i - 100
    @END
    D;JGT   // If (i-100) > 0 goto END
    @i
    D=M     // D = i
    @sum
    M=D+M   // sum += i
    @i
    M=M+1   // i++
    @LLOOP
    0;JMP   // Goto LOOP
(END)
    @END
    0;JMP   // Infinite loop
```

```
0000000000010000
1110111111001000
0000000000010001
1110101010001000
0000000000010000
1111110000010000
0000000001100100
1110010011010000
0000000000010010
1110001100000001
0000000000010000
1111110000010000
0000000000010001
1111000010001000
0000000000010000
1111110111001000
0000000000000100
1110101010000111
0000000000010010
1110101010000111
```

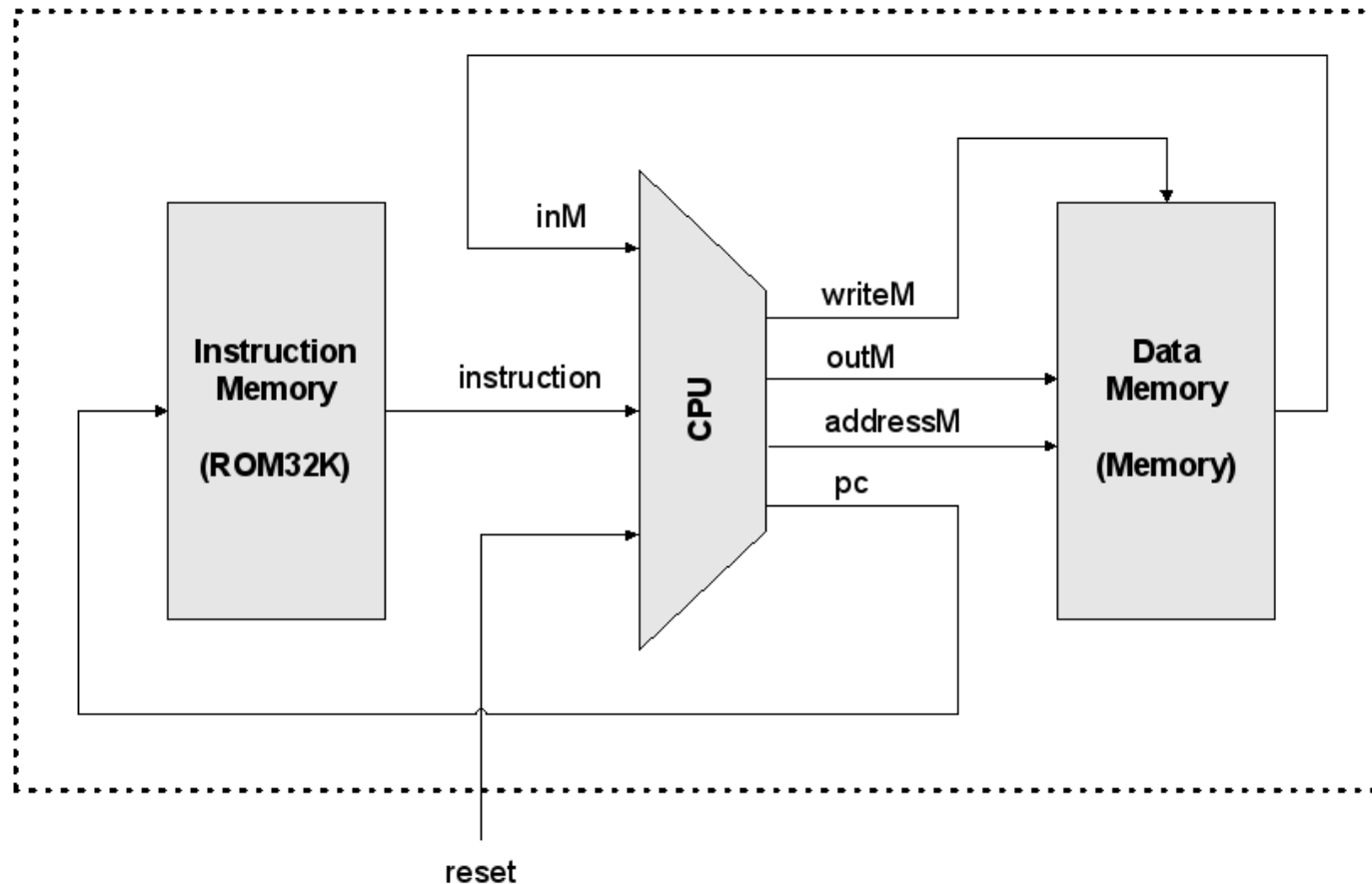
Instruction Set Architecture

- Il linguaggio assembly eseguibile/supportato da uno specifico processore dipende quindi dalla cosiddetta “instruction set architecture” (**ISA**).
- Esiste infatti una **stretta** relazione tra il linguaggio e l’architettura: il linguaggio è progettato per sfruttare l’architettura, e questa è progettata per eseguire il ciclo fetch / decode / execute di istruzioni nel linguaggio.
- Possiamo definire l’**ISA** come l’insieme di quegli elementi dell’architettura che sono “visibili” al compilatore, cioè rilevanti per la compilazione. La compilazione ha il ruolo di generare codice in linguaggio macchina a partire dal linguaggio di alto livello in cui sono scritti i programmi: fanno quindi parte dell’ISA i registri del processore disponibili, il modello di interazione con la memoria, le istruzioni disponibili nell’instruction set, ...
- In questo corso studieremo nel dettaglio l’ISA **del processore Hack**, cominciando dal linguaggio Assembly Hack.
Fra qualche settimana parleremo di nuovo del livello ISA, non solo Hack.

- Storicamente, gli sforzi per migliorare le prestazioni di una architettura hanno portato a due generali approcci di progettazione dell'hardware. Complex Instruction Set Computing (CISC), con cui si cerca di ottenere prestazioni migliori sfruttando un instruction set più elaborato (che quindi permette di scrivere singole istruzioni che potenzialmente richiedono computazioni più complesse), e Reduced Instruction Set Computing (RISC), che utilizzano instruction set in cui ogni singola istruzione è più semplice, con lo scopo di consentire una implementazione hardware ottimizzata.
- Il computer Hack non è facilmente classificabile, dal momento che usa un instruction set semplice (vedi A-instruction e C-instruction più avanti) ma non utilizza tecniche di accelerazione hardware come quelle già studiate.
- Tuttavia, come menzionato nel primo modulo, in Hack l'esecuzione di ogni istruzione viene completata in un singolo ciclo di clock.

Il computer Hack

Comprenderemo nel dettaglio questo diagramma generale, ovviamente completandolo con tutti i dettagli mancanti e necessari per il ciclo Fetch / Decode / Execute delle istruzioni.



Il computer Hack

Hack è una macchina a 16-bit costituita da:

Memoria dati **RAM**: una sequenza di registri a 16 bit che contiene dati

Memoria istruzioni **ROM**: una sequenza di registri a 16 bit che contiene le istruzioni da eseguire, ovvero il programma

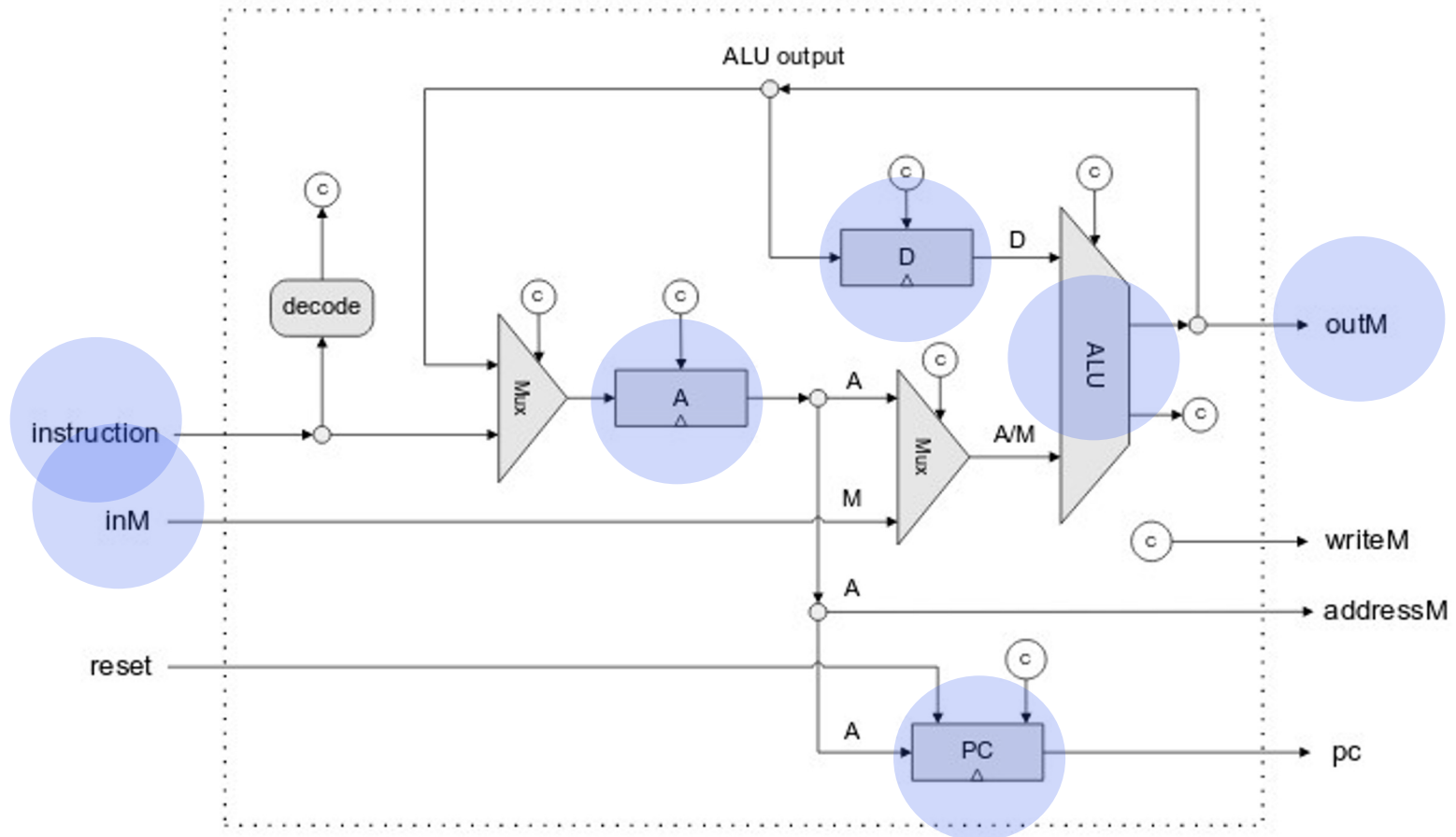
Registri **D** e **A**: i due registri interni del processore
M: il registro di memoria RAM attualmente puntato da A
(cioè all'indirizzo memorizzato correntemente in A)
che denotiamo scrivendo $\text{RAM}[\text{A}]$ – a volte $\text{MEM}[\text{A}]$

ALU: per l'elaborazione, già studiata

Program counter (**PC**): il registro che contiene l'indirizzo del registro contenente la prossima istruzione da eseguire (ovvero in $\text{ROM}[\text{PC}]$). Inizialmente il valore di PC è 0: la prima istruzione è in $\text{ROM}[0]$.

Il computer Hack

I componenti citati nella slide precedente, visti nello schema parziale (incompleto) della CPU Hack. Il componente raffigurato informalmente e chiamato qui «decode» si riferisce alla fase Decode del ciclo di esecuzione, che genera i segnali di controllo marcati come «c» e necessari per la fase Execute (e Fetch dell'istruzione successiva al prossimo ciclo di clock).



Tipi di istruzioni

Esistono due tipi di istruzioni in Assembly HACK:

- **A-instruction:** istruzioni che si limitano a caricare un valore all'interno del registro A. Useremo queste istruzioni per memorizzare in A valori numerici, che useremo per:
 - indirizzi di memoria RAM/ROM (eventualmente rappresentati da un nome: vedi avanti)
 - arbitrari valori
- **C-instruction:** istruzioni che eseguono una computazione (tramite la ALU) prelevando i due operandi dai registri A, D, M, oppure usando le costanti 0, 1 o -1. Il risultato viene memorizzato in A, D, M o combinazioni di questi registri.

Le computazioni che si possono eseguire sono:

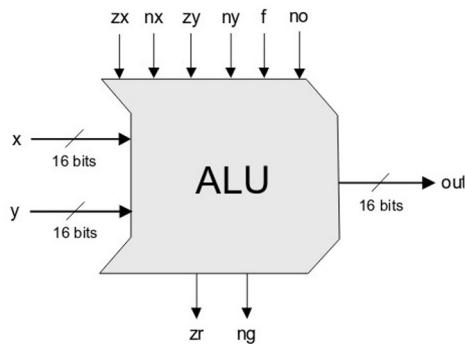
0, 1, -1, D, A, !D, !A, -D, -A, D+1, A+1, D-1, A-1, D+A, D-A, A-D, D&A, D|A, M, !M, -M, M+1, M-1, D+M, D-M, M-D, D&M, D|M

Tipi di istruzioni

Consideriamo ancora le possibili computazioni:

0, 1, -1, D, A, !D, !A, -D, -A, D+1, A+1, D-1, A-1, D+A, D-A, A-D
D&A, D|A, M, !M, -M, M+1, M-1, D+M, D-M, M-D, D&M, D|M

Notare come queste operazioni (che possono prendere come argomenti le costanti 0/1/-1, o il contenuto dei registri A, D, M), sono esattamente le operazioni eseguibili dalla ALU del processore Hack (ripassare di nuovo le slide sulla ALU).



These bits instruct how to preset the x input		These bits instruct how to preset the y input		This bit selects between + / And	This bit inst. how to postset out	Resulting ALU output
zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	f(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x y

Ora vediamo più in dettaglio le A-instruction e C-instruction.

Le C-instruction

Forma generale:

`dest = comp ; jump`

(“`dest =`” e “`; jump`” sono opzionali)

- `comp` specifica la computazione da eseguire
- `dest` specifica dove memorizzare il risultato di tale computazione
- `jump` specifica se, dopo aver eseguito `dest = comp` occorre fare un salto nel programma, ovvero aggiornare il contenuto del PC al valore attualmente salvato nel registro A (così che la prossima istruzione da eseguire non sia la successiva nel programma, ovvero all’indirizzo ROM immediatamente successivo a quello correntemente puntato da PC, cioè eseguendo $PC=PC+1$, ma sia invece l’istruzione salvata nella locazione ROM puntata dal registro A, cioè eseguendo $PC=A$).

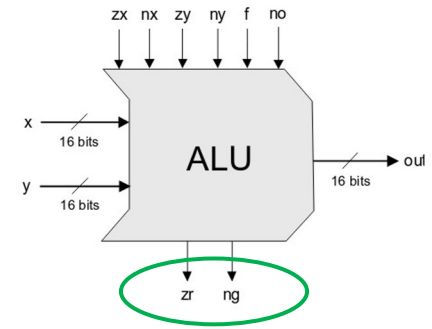
I possibili valori di questi tre elementi:

`comp` = 0, 1, -1, D, A, !D, !A, -D, -A, D+1, A+1, D-1, A-1, D+A, D-A, A-D, D&A, D|A, M, !M, -M, M+1, M-1, D+M, D-M, M-D, D&M, D|M

`dest` = M, D, MD, A, AM, AD, AMD o nullo (in questo caso dest viene omissso)

`jump` = JGT, JEQ, JGE, JLT, JNE, JLE, JMP o nullo (omesso)

Le C-instruction



Capiamo in dettaglio **jump**.

Nell'istruzione della forma “**dest** = **comp** ; **jump**” il salto si esegue controllando l'output della ALU (ovvero il risultato della computazione specificata da **comp**, **confrontandolo con 0** (usando i bit di stato **zr** e **ng** della ALU: vedi slide su ALU).

Le direttive sono: **JGT** per “greater than”, **JEQ** per “equal”, **JGE** per “greater or equal”, **JLT** per “less than”, **JNE** per “not equal”, **JLE** per “less or equal”, **JMP** per “jump always” (nessun test). **Questi sono tutti confronti con 0 !!**

Esempio: **D=D+1 ; JLT**

Questa istruzione corrisponde ad incrementare di 1 il contenuto del registro D, ovvero calcolare $D=D+1$ e, successivamente, se $(D+1) < 0$ saltare nel programma all'istruzione che è memorizzata all'indirizzo ROM pari al valore corrente del registro A.

- Se **jump** non è nullo, di solito si testa il valore di D e non M/A (sai dire perché?)
- Un salto incondizionato (non c'è test da fare) si codifica con 0 ; **JMP**

Le A-instruction

Forma generale:

```
@value // A = value
```

Dove *value* è un numero

(...o un nome che rappresenta un numero, vedi dopo)

Esempi pratici di utilizzo:

- Caricare una costante da utilizzare in altre istruzioni. Esempio: D = **value**
- Leggere un dato dalla **RAM**
Per esempio D = **RAM[A]**
- Selezionare una locazione **ROM** per effettuare un salto nel programma: **PC = A**

Istruzioni in Assembly Hack:

```
@17 // A = 17
```

```
D = A // D = 17
```

C-instruction

```
@17 // A = 17
```

```
D = M // D = RAM[17]
```

C-instruction

```
@17 // A = 17
```

```
0 ; JMP // salva 17 in PC, quindi la prossima  
// istruzione sarà ROM[17]
```

C-instruction

Iniziamo a programmare in Assembly Hack

Esercizio: per ciascuno dei seguenti semplici “programmi” scritti in pseudocodice qui sotto, scrivere un possibile programma in Assembly Hack corrispondente.

1. // D=0

6. // D=3+4

2. // A=0

3. // A=1
// D=1

7. // RAM[3]=7

4. // D=3

8. // D=RAM[A]+3

5. // D=RAM[3]+3

Iniziamo a programmare in Assembly Hack

Esercizio: per ciascuno dei seguenti semplici “programmi” scritti in pseudocodice qui sotto, scrivere un possibile programma in Assembly Hack corrispondente.

1. // D=0

```
D=0
```

2. // A=0

```
@0 // nota: C-instruction A=-1/0/1 ok
```

3. // A=1

```
// D=1
```

```
AD=1 // o anche @1 e poi D=1
```

4. // D=3

```
@3
```

```
D=A
```

5. // D=RAM[3]+3

```
@3
```

```
D=M D=D+A
```

6. // D=3+4

```
@3
```

```
D=A
```

```
@4
```

```
D=D+A
```

7. // RAM[3]=7

```
@7
```

```
D=A // prima il dato da scrivere
```

```
@3 // e poi dove scriverlo
```

```
M=D
```

8. // D=RAM[A]+3

```
D=M
```

```
@3
```

```
D=D+A
```

Iniziamo a programmare in Assembly Hack

Altri semplici esercizi.

1. // D=D-1

2. // D=D-3

3. // D=10-RAM[5]

4. // RAM[0]=RAM[0]+2

6. // D=RAM[RAM[0]]

7. // RAM[RAM[0]] = RAM[0]

Iniziamo a programmare in Assembly Hack

Altri semplici esercizi.

1. // D=D-1

```
D=D-1
```

2. // D=D-3

```
@3
```

```
D=D-A
```

3. // D=10-RAM[5]

```
@10
```

```
D=A
```

```
@5
```

```
D=D-M
```

```
@5
```

```
D=M
```

```
@10
```

```
D=A-D
```

oppure

4. // RAM[0]=RAM[0]+2

```
@2
```

```
D=A // prima il dato da sommare
```

```
@0 // e poi dove scriverlo
```

```
M=M+D
```

6. // D=RAM[RAM[0]]

```
@0
```

```
A=M
```

```
D=M
```

// diverso da AD=M !!

Ricordarsi: quando **dest** specifica più di un registro, l'assegnazione è contemporanea! Quindi qui AD=M equivarrebbe a calcolare A=RAM[0] e D=RAM[0].

Invece vogliamo prima leggere un indirizzo da RAM[0], e poi usare quell'indirizzo per la seconda lettura.

7. // RAM[RAM[0]] = RAM[0]

```
@0
```

```
D=M // l'ordine non si può invertire, e
```

```
A=M // anche solo AD=M sarebbe ok
```

```
M=D
```

Iniziamo a programmare in Assembly Hack

Vediamo un esercizio leggermente più complesso.

Due modi per salvare in RAM[2] la somma RAM[0]+RAM[1]

// RAM[2]=RAM[0]+RAM[1]

```
D=0           // in caso D non sia uguale a 0
@0
D=D+M         // D += RAM[0]
@1
D=D+M         // D += RAM[1]
@2
M=D           // RAM[2]=D
```

// RAM[2]=RAM[0]+RAM[1]

```
@2           // in caso RAM[2] non sia 0:
M=0           // RAM[2]=0
@0
D=M           // D=RAM[0]
@2
M=D+M         // RAM[2] += D
@1
D=M           // D=RAM[1]
@2
M=D+M         // RAM[2] += D
```

Nota: nei commenti usiamo l'abbreviazione $X+=Y$ al posto di $X=X+Y$, e similmente $X-=Y$ per $X=X-Y$.

Iniziamo a programmare in Assembly Hack

Altri semplici esercizi con salti.

1. // Se $\text{RAM}[0] > 0$, goto istruzione con indirizzo memorizzato in $\text{RAM}[1]$

2. // $D = D + 1$; successivamente, se $D = 0$ goto istruzione 3

3. // $\text{RAM}[0] = \text{RAM}[5]$ e se $\text{RAM}[5] \neq 0$ goto istruzione 3

Iniziamo a programmare in Assembly Hack

Altri semplici esercizi con salti.

1. // Se $\text{RAM}[0] > 0$, goto istruzione con indirizzo memorizzato in $\text{RAM}[1]$

@0
D=M

(continua a destra:)

@1
A=M
D ; JGT

2. // $D = D + 1$; successivamente, se $D = 0$ goto istruzione 3

@3
D=D+1 ; JEQ

3. // $\text{RAM}[0] = \text{RAM}[5]$ e se $\text{RAM}[5] \neq 0$ goto istruzione 3

@5
D=M

@0
(continua a destra:)

M=D
@3
D ; JNE

Etichette

Oltre alle A/C-instruction, possiamo definire **etichette**, ed usarle nelle A-instruction come fossero valori (riguarda ora la slide sulle A-instruction).

```
0:  @2
1:  D=A
   (BACK)
2:  @4
3:  D=M
4:  D=D+M
5:  @2
6:  D=D+A
7:  @BACK
8:  D ; JGE
```

Dichiarazione di etichetta:

dichiariamo un nome a piacimento per far riferimento all'indirizzo dell'istruzione successiva al punto di dichiarazione. In questo caso "BACK" vale 2 e quindi fa riferimento all'istruzione in ROM[2], ovvero "@4".

Uso di etichetta:

il simbolo "BACK" sarà sostituito (studieremo poi da chi e quando) con il valore dell'etichetta, cioè 2. Quindi nell'istruzione successiva, se il salto avverrà, sarà un salto all'istruzione in ROM[2] cioè "@4". Quindi invece di @BACK potremmo scrivere direttamente @2, ma le etichette ci evitano di dover gestire esplicitamente i numeri di riga (quindi indirizzi ROM) nei nostri programmi.

L'indentazione del programma è irrilevante

Flusso di controllo del programma: costruito “if-then-else”

Possiamo utilizzare il **costrutto if-then-else**, solitamente disponibile in linguaggi di alto livello, sfruttando i salti delle C-instruction e le etichette.

```
if (condizione es: D-1>0) {  
    ...blocco di codice 1...  
}  
else {  
    ...blocco di codice 2...  
}  
...blocco di codice 3...
```

```
D=D-1      // o qualunque sia il dato da testare  
@CASO_FALSE // preparo destinazione salto  
D; JLE      // salto se D<=0, altrimenti:  
...blocco di codice 1...  
@END  
0; JMP      // salto incondizionato  
(CASO_FALSE)  
...blocco di codice 2...  
(END)  
...blocco di codice 3...
```

Flusso di controllo del programma: costruito “while”

Idem per il **costrutto while**. Da notare che possiamo implementarlo sia come while-do sia come do-while. Qui vediamo il while-do. Per esercizio, implementa il do-while (il test va alla fine del corpo del ciclo, non all’inizio).

```
while (condizione es: D>0) {  
    ...blocco di codice 1...  
}  
...blocco di codice 2...
```

```
(LOOP)  
  
    @EXIT  
  
    D ; JLE                // salto se D<=0  
    ...blocco di codice 1...  
  
    @LOOP  
  
    0 ; JMP                // ricomincia ciclo  
  
(EXIT)  
  
    ...blocco di codice 2...
```

Iniziamo a programmare Hack

Esempio: $\text{RAM}[2] = \text{RAM}[0] \times \text{RAM}[1]$

// assumendo $\text{RAM}[1] > 0$

Implementiamolo come $\text{RAM}[2] = \text{RAM}[0] + \dots + \text{RAM}[0]$ ($\text{RAM}[1]$ volte)

```
@2           // inizializzazione:
M=0          // RAM[2]=0
(LOOP)
@0
D=M          // D=RAM[0]
@2
M=D+M        // RAM[2] += D, ovvero RAM[2] += RAM[0]
@1
MD=M-1       // RAM[1]=RAM[1]-1 : usiamo RAM[1] come contatore dei cicli
@LOOP
D ; JGT      // ricominciare il ciclo se RAM[1]>0
```

Nota: questo è un “do-while” perché $\text{RAM}[1] > 0$ quindi il ciclo va eseguito almeno una volta.

MD=M-1

Ricorda: queste assegnazioni sono concorrenti: $M=M-1$ e allo stesso tempo $D=M-1$. Avremmo potuto usare due istruzioni (in quale ordine?)

Iniziamo a programmare Hack

Esercizio difficile: $\text{RAM}[0..9] = [10, 9, 8, \dots, 1]$ ovvero $\text{RAM}[0]=10, \text{RAM}[1]=9, \dots$

```
@10
M=-1          // RAM[10]=-1   variabile indirizzo 0..9
@10
D=A
@11
M=D           // RAM[11]=10   variabile numero 10..1 da scrivere
(LOOP)        // do-
@11
D=M           // D=RAM[11]      D=numero da scrivere
M=M-1         // RAM[11] -= 1   decremento
@10
AM=M+1        // A = RAM[10]+1, RAM[10]+=1  indirizzo da scrivere
M=D           // RAM[A] = D      scrivo D
@11
D=M
@LOOP
D; JGT        // -while: se RAM[11]>0 ricomincia ciclo
```

Fine dei programmi

Nota: di default, non esiste una “fine” del programma. Se non facessimo nulla per impedirlo, il PC verrebbe incrementato per sempre ogni volta che il processore ha finito di eseguire l’istruzione corrente (ovviamente, “per sempre” qui significa fino all’ultimo indirizzo della ROM...).

Per questo motivo è buona norma terminare ogni programma con un ciclo dummy che “previene” l’esecuzione di istruzioni scritte in locazioni ROM successive (e che in generale potrebbero appartenere ad un altro programma).

```
...  
...  
(END)           // ciclo dummy finale  
  @END  
  0 ; JMP
```

Osservazione: comparazioni NON per 0

Nelle C-instruction scritte negli esempi fatti finora, abbiamo sempre avuto bisogno di comparare per 0 per stabilire se eseguire un salto.

E, come visto, il linguaggio Assembly Hack considera sempre test per 0.

Ovviamente, questo non vuol dire che non possiamo comparare con valori arbitrari diversi da 0: prima di eseguire il test del salto, usiamo addizioni/sottrazioni nel modo opportuno.

Pseudocodice di esempio: // if RAM[2]>3 then RAM[5]=1 else RAM[5]=0

Un possibile programma (ne possiamo immaginare svariati):

```
@2
D=M
@3
D=D-A      // D=RAM[2]-3
@ELSE
D ; JLE    // salta se D<=0, ovvero RAM[2]<=3
@5
M=1
```

```
@END
0 ; JMP
(ELSE)
@5
M=0
(END)
@END
0 ; JMP
```

Iniziamo a programmare Hack

Esercizio: $\text{RAM}[2] = \max(\text{RAM}[0], \text{RAM}[1])$

```
@0
D=M           // D=RAM[0]
@1
D=D-M         // D -= RAM[1]
@SECOND
D ; JLT        // salta se RAM[1]>RAM[0]
               oppure JLE
@0
D=M
@2
M=D           // RAM[2]=RAM[0]
@END
0 ; JMP        // goto END
```

```
(SECOND)
@1
D=M
@2
M=D           // RAM[2]=RAM[1]

(END)         // loop finale
@END
0 ; JMP
```

Iniziamo a programmare Hack

Esercizio: $\text{RAM}[10] = \max(\text{RAM}[9], \text{RAM}[8], \dots, \text{RAM}[1], \text{RAM}[0])$

```
// RAM[10] = max(RAM[9..0])
```

```
@9
D=M           // D=RAM[9]
@10
M=D           // RAM[10]=D
@8
D=A
@11
M=D           // RAM[11]=8
(LOOP)        // do...
@11
A=M
D=M           // D=RAM[RAM[11]]
@10
D=D-M         // D -= RAM[10]
```

```
@NEXT
D ; JLE       // salta se (D <= 0)
@11
A=M
D=M
@10
M=D           // RAM[10]=RAM[RAM[11]]
(NEXT)
@11
MD=M-1        // RAM[11]-=1
@LOOP
D ; JGE       // ...while RAM[11]>=0

(END)
@END
0 ; JMP
```

Nota: stiamo usando RAM[11] come contatore cicli, mentre in RAM[10] teniamo il corrente massimo valore trovato.

Ulteriori esercizi

Scrivere dei programmi HACK che rispondano alle specifiche seguenti

1. $RAM[2] = RAM[1] - RAM[0] - 2$
2. $RAM[2] = RAM[1] \text{ nand } RAM[0]$ //nand bit a bit
3. Scrivere il valore 1 in tutte le celle di memoria con indirizzo compreso tra $RAM[0]$ e $RAM[1]$, assumendo $RAM[1] > RAM[0] > 1$
4. $RAM[2] = 2 \times RAM[2]$ // qualche slide più su abbiamo già visto la moltiplicazione arbitraria. In questo caso, basta qualcosa di più semplice?
5. $RAM[2] = RAM[1] \times (2^{RAM[0]})$
6. $RAM[2] = RAM[1] / RAM[0]$, $RAM[3] = RAM[1] \bmod RAM[0]$ //divisione intera

Per il momento possiamo controllare la correttezza di un programma solo a mano. Ben presto useremo un emulatore del computer Hack.

Uso dei simboli nel linguaggio Hack

In Hack si usano due tipi di simboli:

- **Etichette:** Già viste, usate per fare riferimento ad indirizzi della **ROM**. Dichiarate tramite direttiva (**XXX**) che definisce il simbolo **XXX** che farà riferimento all'indirizzo di ROM dell'istruzione successiva alla dichiarazione
- **Variabili:** Usate per far riferimento ad indirizzi in memoria **RAM**. Ci sono due tipi di variabili:

pre-definite: ad esempio **SCREEN** e **KBD** che fanno riferimento agli indirizzi RAM 16384 e 24576 (indicano rispettivamente l'inizio della memoria per gestire lo schermo e la locazione dove viene inserito il codice di un eventuale tasto premuto su tastiera).

definite dall'utente: un simbolo non predefinito **xxx** usato in un programma Hack senza essere dichiarato usando (**xxx**) viene trattato come variabile, e gli viene assegnato in automatico un valore (a partire dal valore 16: indirizzi da 0 a 15 sono usati dalle variabili predefinite **R0..R15**). Ne discuteremo in futuro.

Chi assegna i valori ai simboli? **L'assemblatore**, cioè il programma che traduce un programma Hack simbolico in binario (studieremo più avanti)

// esempio (il significato non è importante):

```
@R0
D=M
@INFINITE_LOOP
D;JLE
@counter
M=D
@SCREEN
D=A
@addr
M=D
(LOOP)
@addr
A=M
M=-1
@addr
D=M
@32
D=D+A
@addr
M=D
@counter
MD=M-1
@LOOP
D;JGT
(END_LOOP)
@END_LOOP
0;JMP
```

Esempio di esercizio usando simboli (variabili ed etichette)

Per questo esercizio,
atteniamoci al seguente
linguaggio C / pseudocodice:

```
// Somma 1+...+100.  
int i = 1;  
int sum = 0;  
while (i <= 100){  
    sum += i;  
    i++;  
}
```

Convenzione sui simboli:

Variabili: **minuscolo**

Etichette: **maiuscolo**

Ora modifica soluzioni di
esercizi già svolti, questa
volta usando variabili.

Assembly Hack:

```
@i                // variabile i (salvata in qualche indirizzo)  
M=1              // i=1  
@sum             // variabile sum (come sopra)  
M=0              // sum=0  
(LOOP)  
@i  
D=M              // D = i  
@100  
D=D-A            // D = i - 100  
@END  
D;JGT            // se (i-100) > 0 salta ad END  
@i  
D=M              // D = i  
@sum  
M=D+M            // sum += i  
@i  
M=M+1            // i++  
@LOOP  
0;JMP            // ricomincio il ciclo  
(END)  
@END  
0;JMP
```


Nota sull'uso delle variabili

Esercitarsi a fondo sulla scrittura di programmi in Assembly Hack. Solo dopo, considerare il contenuto di questa slide.

Abbiamo detto (due slide sopra) che esistono variabili predefinite e che queste fanno riferimento a (cioè il loro valore viene memorizzato in) locazioni di memoria predefinite. Per esempio le variabili **R0..R15** fanno riferimento a RAM[0]...RAM[15]. Infatti scrivere @R0 equivale a scrivere @0. In termini pratici, le locazioni di memoria in questo intervallo sono riservate per essere utilizzate a piacimento dal programma in esecuzione, come memoria di lavoro.

Non possiamo sapere invece in quali locazioni di memoria verranno memorizzate le variabili non predefinite ma definite da noi: l'assemblatore, durante la conversione da codice simbolico a codice binario, fisserà questa scelta (studieremo l'assemblatore più avanti).

Per ora sappiamo solo che le variabili non predefinite fanno riferimento a locazioni dal 16 in poi.

Per questo motivo, se per risolvere un esercizio ci troviamo a dover scrivere su RAM[16], RAM[17], etc..., ovvero includiamo istruzioni come

...

@16

M=0

...

allora non dovremmo usare variabili nel nostro programma, perché corriamo il rischio di sovrascriverle accidentalmente. Per esempio, considera l'esercizio 3 nella slide "Ulteriori esercizi".